As with JBehave, Cucumber expects the step definition classes to be in a package below the test runner class.

6.5.2 Cucumber-JVM step definitions

Let's see how you could use Cucumber-JVM to automate the following scenario:

```
Given the flying distance between Sydney and Melbourne is 878 km
And I am a Gold Frequent Flyer member
When I fly from Sydney to Melbourne
Then I should earn 878 travel points
```

Cucumber-JVM step definitions are annotated Java methods, similar to the ones used by JBehave:[11]

```
import cucumber.api.java.en.Given;
...
@Given("I am a Gold Frequent Flyer member")
public void useAMemberWithGoldStatus() {
    // TODO: use a member with a Gold status
}
```

Note that the Cucumber annotations come from a different package than the JBehave ones.

Unlike JBehave, Cucumber doesn't mind which annotation you use: `@Given`, `@When`, and `@Then` are treated as synonyms.

As you've seen, the Gherkin language used by Cucumber relies on regular expressions to identify variables in the steps. For example, if you want to pass the `status` to the step definition just discussed, you could write something like this:

The (*.) regular expression will be passed in to the status parameter.

```
@Given("^I am a (.*) Frequent Flyer member$")
public void useAMemberWithAGivenStatus(String status) {
    // TODO: use a member with a Gold status
}
```

Regular expressions appear in parentheses. The ^ symbol indicates the start of a line, and the $ symbol indicates the end of a line.

Cucumber will match the regular expressions in parentheses and assign them to the method parameters. In the following step definition, you pass in three parameters:

Here you have three regular expressions.

```
@When("^I fly from (.*) to (.*) on (.*)$")
public void I_fly_from(String departure,
                       String destination,
                       Date date){
    // TODO
}
```

Matches the first (.*).

Matches the second (.*).

Matches the third (.*).

Cucumber will convert many parameter types for you. In this step definition, for example, the third parameter will be converted to a `Date`.

6.5.3 Pattern variants

The use of regular expressions adds a great deal of power to step definitions, and it's easy to define several variations of the same step. But there are a few tricks to know.

[11] In fact, the only difference here is that the `@Given` annotation comes from a different package.